MEZO BUILDATHON / MUO

# MUSD-backed optimistic claims.

## Mezo Upbeat Oracle

Put a bond behind a claim. Let others challenge it.
Settle on Mezo with MUSD.

Contract live on Mezo Testnet / Chain ID 31611

# Price feeds do not settle every claim.

A contract can read a BTC price. It cannot directly determine whether an external statement is true.

## Already on Mezo

- Skip Connect supplies BTC/USD data.
- Pyth supplies additional price feeds.
- MUSD provides Bitcoin-backed USD liquidity.

## Potential claim use cases

- Reserve statements
- Event outcomes
- Protocol parameter proposals

These are target use cases, not live customer integrations. MUO complements price feeds; it does not replace them.

# Assert. Challenge. Settle.

An economic challenge process, backed by MUSD.

| | |
|---|---|
| **Assert** | Lock at least 10 whole MUSD behind a claim of 1 to 2,048 bytes. |
| **Wait** | The window is 1 to 52,560 blocks. Input 0 selects 1,440 blocks. |
| **Settle** | After the window closes with no dispute, anyone can return the bond to the asserter. |
| **Dispute** | Another party locks a matching bond while the window is open. |
| **Resolve** | An approved arbiter, who is neither party, selects outcome 2, 3 or 4. |

**0 Open / 1 Disputed / 2 Settled / 3 Rejected / 4 Unresolved**

Dispute outcomes: 2 pays both bonds to the asserter; 3 pays both to the disputer; 4 refunds one bond to each.

# One assertion, step by step

| | | |
|---|---|---|
| 01 | Owner | Deploy MuoCore with the MUSD token address. |
| 02 | Owner | Add an arbiter if needed; the deployer starts as an arbiter. |
| 03 | Asserter | Sign a permit or approve MUSD spending. |
| 04 | Asserter | Call assertClaim or assertWithPermit. |
| 05 | MuoCore | Check the claim, bond and liveness window. |
| 06 | MuoCore | Lock the asserter's MUSD bond. |
| 07 | MuoCore | Emit Asserted; status becomes Open (0). |
| 08 | Observers | Check the claim during the window. |
| 09 | Anyone | Settle after the window closes if there is no dispute. |
| 10 | Disputer | Or dispute while open and lock a matching bond. |
| 11 | Arbiter | Resolve a disputed claim with outcome 2, 3 or 4. |
| 12 | MuoCore | Pay the winner or refund both parties. |

# MUSD is the bond and the payout.

No new oracle token is required.

## MUSD bonding

Bonds use whole MUSD as input. The contract transfers that amount times 10^18 token units through transferFrom.

## Permit support

assertWithPermit and disputeWithPermit use EIP-2612 to avoid a separate approval transaction. This is contract functionality, not yet a completed portal flow.

## Skip Connect

getBtcPrice returns the feed's answer and updatedAt timestamp. Price data does not automatically verify arbitrary claims.

Current arbitration uses an owner-managed allowlist. Stake-weighted arbitration is planned.

# Live contract. Checkable evidence.

| | |
|---|---|
| **Network** | Mezo Testnet / chain ID 31611 |
| **MuoCore** | 0x61c41cD7299fD668c3a7aAE62F12c017fE654Cb0 |
| **Deployment block** | 16,380,473 |
| **Deployment tx** | 0x54e2772177c370b1fe1608b88b5795fdce8cf0ae91453a59c583663c95b02916 |
| **MUSD token** | 0x118917a40FAF1CD7a13dB0Ef56C86De7973Ac503 |
| **Build** | Solidity 0.8.28 / London EVM / optimizer 200 runs |
| **Runtime** | 7,440 bytes |
| **Recorded events** | 3 Asserted / 1 Disputed / 2 Settled / 1 ArbiterAdded |
| **Live API** | muo-api.gborlahg.workers.dev/health |

Event counts are not transaction counts. Explorer source verification is pending. MUO is not deployed on mainnet.

# 92 automated tests pass.

## 40 API tests

Hono / Cloudflare Workers / D1

## 52 portal tests

Next.js / React / TypeScript

### A separate Workers runtime smoke test

Runs the minified production bundle. Checks D1 writes and reads, CORS, preflight requests and webhook authentication.

### Native tooling

Wrangler 4.153.0 / matching Workers types / startup profiling / API TypeScript and Biome checks pass.

Worker bundle: 87.50 KiB raw / 30.25 KiB gzip. These are build sizes, not latency measurements.

No contract test suite is included yet. No new coverage percentage is claimed.

# From contract to usable oracle.

1. Connect Mezo EVM event ingestion to the live API.
2. Add Mezo wallet signing and permit flows.
3. Test complete assert, dispute and resolve flows in the portal.
4. Add contract tests and verify the deployed source.
5. Develop stake-weighted arbitration before mainnet.

**The current portal is not an end-to-end Mezo transaction demo.**

Landing site: mezo-upbeat-oracle-site.vercel.app

Intended repository: github.com/symulacr/muo (publication pending)

## MUSD-backed claims. Public evidence. Clear limits.